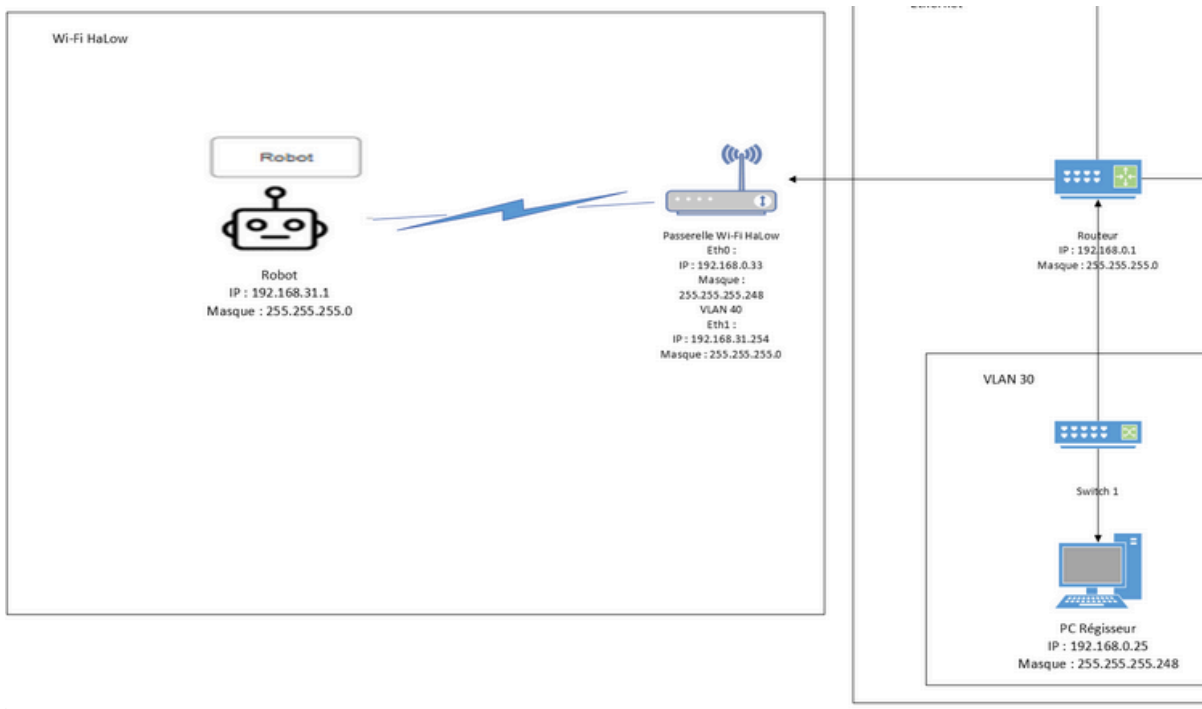


1. Introduction

Ce document va permettre d'expliquer comment va communiquer l'application hébergé sur le PC du régisseur et le robot

Architecture réseau représentant les différents appareils :



2. Partie connexion au robot

Etant donné que la communication entre le robot et l'application se fait en TCP, avant de transmettre les données au robot l'application devra donc se connecter au robot. Pour cela l'application utilise la méthode ConnexionRobot qui utilise la méthode ClientTCP::Connect()

Cette méthode permet de se connecter en TCP à l'adresse et au port passé en paramètre dans la méthode

```
public string ConnexionRobot(string adresse)
{
    string resultat = "";

    //Se connecte au serveur
    if (client.Connected == false)
    {
        client.Connect(adresse, 88); //Connection sur le serveur
        stream = client.GetStream(); //Flux ou l'on va envoyé les données
        resultat = "connection au serveur reussit";
    }
    else
    {
        throw new SocketException(1); //???
    }

    return resultat;
}
```

3. Partie transmission des données

3.1. Spécification

Nous avons défini la trame d'envoi des données des robots comme étant celle-ci :

List<Longitude, latitude, etat, distance>	count
---	-------

Afin de simplifier la manipulation des données nous avons décidé de stocker les informations nécessaire au robot dans une structure appelé DonnéesRobot

ici la liste contiendra des objets de DonnéesRobot contenant donc :

- Le point de mesure
- etat (bit représentant si c'est un point de trajectoire ou non)
- la distance comparé au précédent point

donc la trame devient comme ceci :

Liste de DonnéesRobot	count
-----------------------	-------

3.2. Conversion des données en octets

Code correspondant au sein de la méthode

```
var int_lat = Math.Round(données_robot[i].point_mesure.Lat, 5) * 100000;
var int_lng = Math.Round(données_robot[i].point_mesure.Lng, 5) * 100000;

//Convertie les int en 4 octets
var byte_lat = BitConverter.GetBytes((int)int_lat);
var byte_lng = BitConverter.GetBytes((int)int_lng);
var byte_distance = BitConverter.GetBytes((short)données_robot[i].distance);

//copie des octets dans message
var byte_message = byte_lat.Concat(byte_lng).ToArray().Concat(byte_distance).ToArray();
message = message.Concat(byte_message).ToArray();

//ajout du count a la fin
var byte_count = BitConverter.GetBytes((short)données_robot.Count);
message = message.Concat(byte_count).ToArray();
```

Afin de transmettre les coordonnées au robot, l'application doit d'abord convertir ces données en octets avant de les envoyés

Documentation méthode EnvoiePointDeMesure

Pour cela l'application procède en plusieurs étapes :

- Tout d'abord l'application arrondit les coordonnées GPS à 5 décimal afin de limiter l'espace pris dans la bande passante (Math.Round())
- Ensuite elle va multiplier ces coordonnées par 100000 afin d'enlever la virgule et de faire passer les coordonnées d'un type double à un type int ce qui permet aussi de libérer de la bande passante
- Enfin elle convertit tout en octet (méthode BitConverter.GetBytes())

3.3. Envoi des données :

Code correspondant au sein de la méthode

```
//Envoi du message
if (stream.CanWrite == true)
{
    //Une fois que l'on aura fait le calcul de traj mettre la distance entre les points avec la latitude et la longitude (comme dit dans le doc)
    stream.Write(message, 0, message.Length);
    resultat = "Les points de mesures ont bien été envoyé";
}
```

L'application utilise la méthode `NetworkStream::Write()` afin d'envoyer les données

Cette méthode envoie le tableau d'octet passé en paramètre dans une requête TCP via le flux `NetworkStream` créé précédemment

Tous les points seront donc envoyés d'un seul coup au robot